



Quality Analytics Case Study

Test-bay failure rate reduction - data pipeline, SPC, and executive delivery



About this project

Domain-first, not tool-first.

Built by a quality and operations professional using a data pipeline to solve a business problem I have lived with. The analytical stack (SQL, Python, Power BI) is a means to an end - the end is a defensible view of quality performance that leadership can act on.

ANONYMISATION NOTICE

Vortek Appliances is a fictional alias. All names, IDs and identifiers have been transposed from a real UK-based commercial-appliance manufacturer to protect commercial confidentiality. The failure modes, workflow and interventions are faithful to reality.

Quality Engineering

Pareto - SPC - MTTR - COPQ

Data Engineering

MySQL - pipeline design - star schema

Analytics and BI

Python - DAX - Power BI - storytelling



The business problem

One in three units failing first-time test.

~30%

Test-bay failure rate

£20K

Peak monthly rework cost

350+

Peak monthly rework hours

0

Structured pipeline in place

The ask from leadership

FD

Finance - visibility on rework cost, run rate, and impact of change

OD

Ops Director - pipeline the team can trust, not spreadsheets and gut feel

TD

Technical Director - root-cause detail behind the failure numbers

MD

MD - evidence the intervention programme is delivering results



Approach - SQL to Python to Power BI

Each stage had a job. The pipeline was designed, not stitched together.

1

SQL - foundation

Loaded raw test data into MySQL. Initial exploratory queries, cleaning, column drops. Repeatable, server-side, not a one-off script.

2

Python - discovery

Pass and fail split, fault frequency, monthly trend, cost and hours impact, breakdowns by unit and assembler. Shape of the problem.

3

Python - Pareto and p-chart

Crystallisation. Which faults matter. Is the process in statistical control. This is where dashboard design fell out.

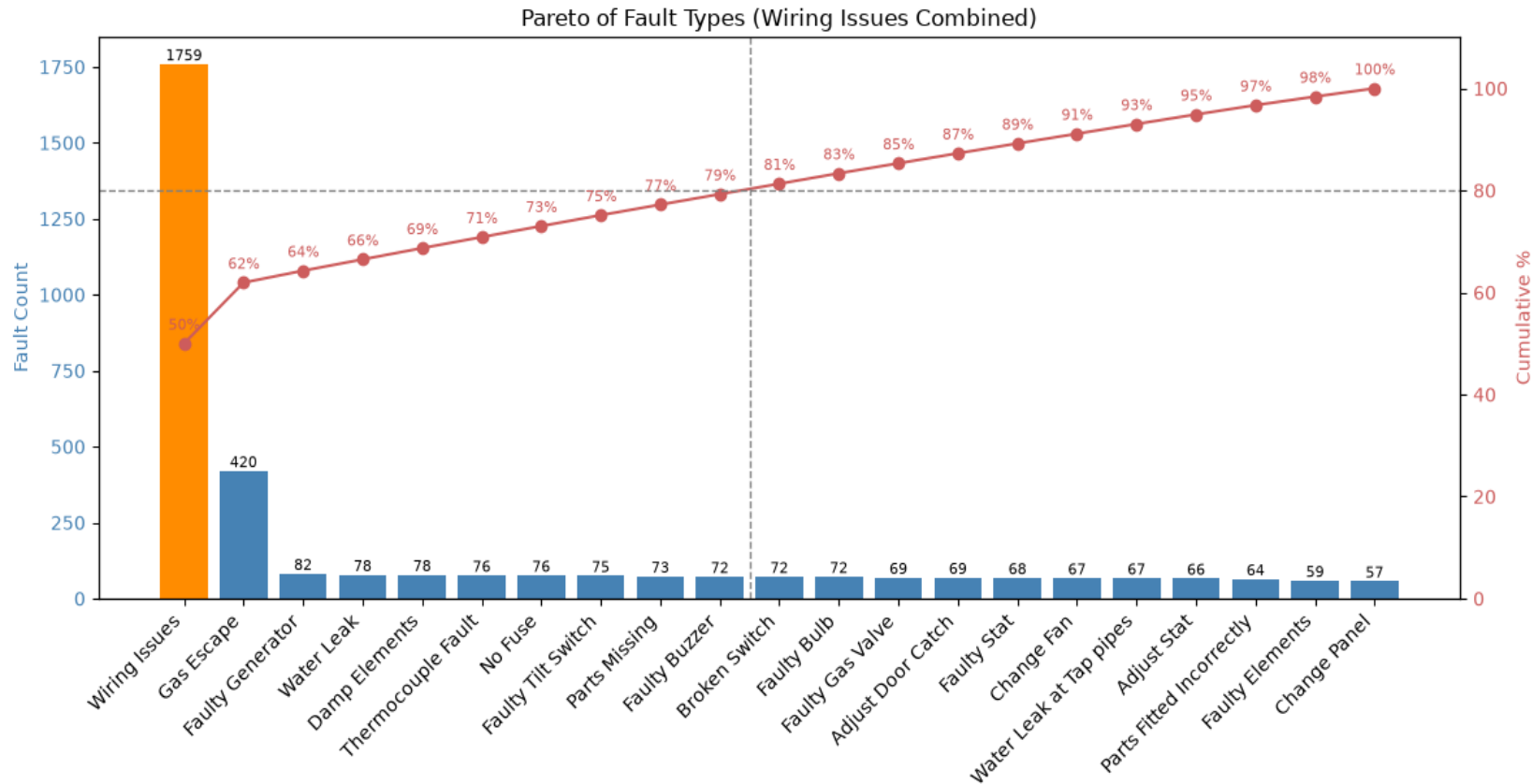
4

Power BI - delivery

Live connection to MySQL. Power Query cleaning. DAX measures for Failure Rate, MTTR, COPQ, and pre/post. C-suite dashboards.



Discovery - Pareto: wiring dominates



80/20 cutoff = 10 fault categories. Half of those 10 = wiring alone.

KEY FINDING

Wiring Issues = 1,759 faults

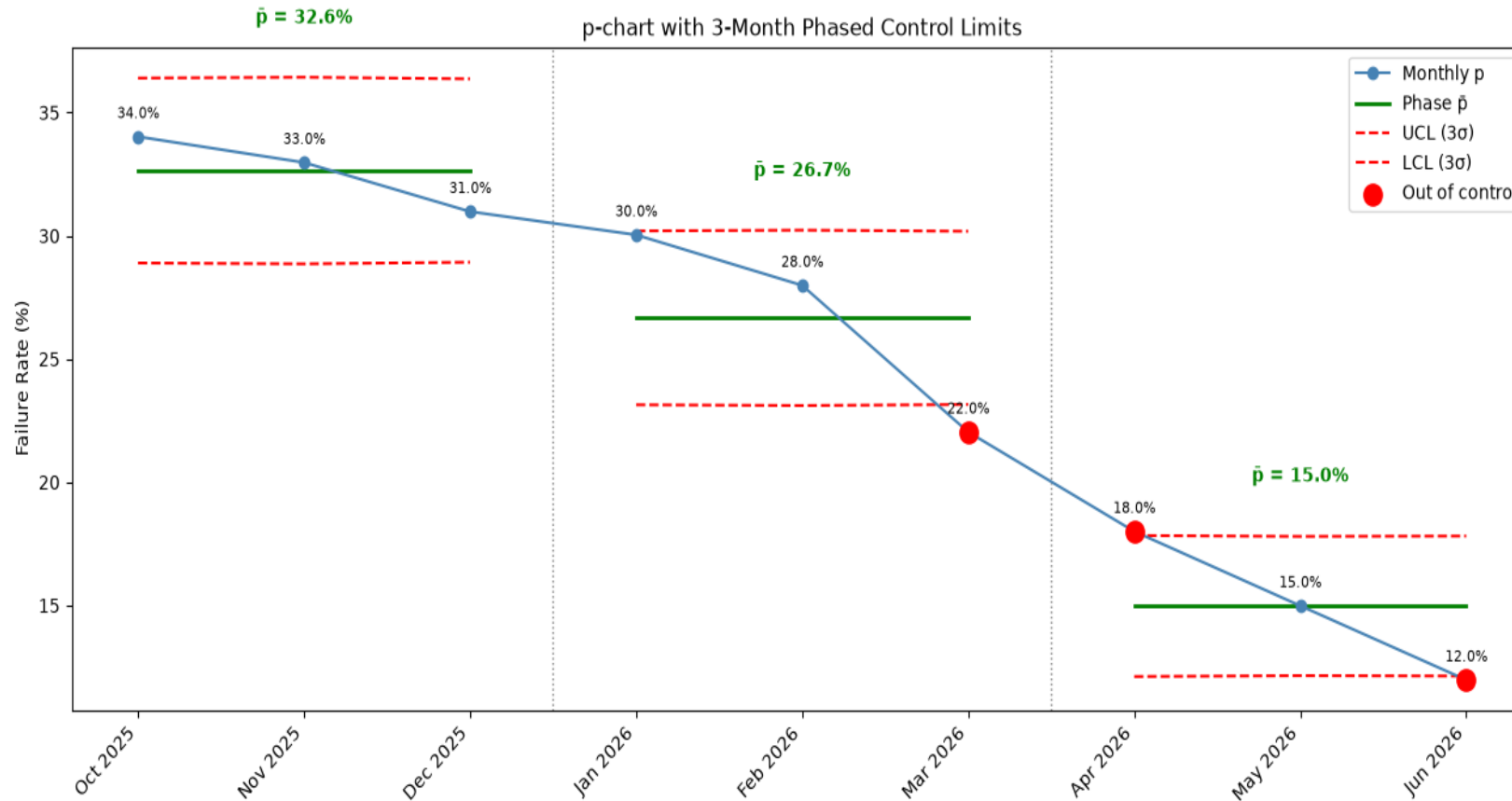
Roughly 50 percent of all faults once the five wiring failure modes are consolidated. Gas Escape sits second at 420.

SO WHAT

Attack wiring at the assembly step (feedback loop, procedure, training) and you neutralise half the problem in one intervention.



Discovery - p-chart: process is not in control



Three consecutive months above UCL pre-intervention. Three below LCL post. This is a step-change, not noise.

PHASE 1 (pre)

32.6%

Oct - Dec 2025

PHASE 2 (transition)

26.7%

Jan - Mar 2026

PHASE 3 (post)

15.0%

Apr - Jun 2026



Root causes - beyond the numbers

Two mechanisms found in the plant that map to the two biggest Pareto bars.

1

Training gap

Assembly - safety-critical station

Operator recently transferred without full training sign-off. Skills-matrix control failed.

> Explains Gas Escape spikes.

2

Procedure drift

Test bay - standard-work violation

Tester executing beyond the documented test procedure. Introduced variability the process was not designed for.

> Amplified Gas Escape spikes.

3

Wiring workmanship

Assembly - routing and connection

Five failure modes: loose, trapped, not-fitted, broken, incorrectly wired. Same underlying category.

> Explains wiring dominance.



Interventions - mapped to root causes

Every change addressed a specific mechanism. AI was the enabler, not the cause.

HIGH

Wiring feedback loop

Test-bench findings routed daily to assembly. Attacks the number one Pareto bar.

HIGH

Updated build procedures

Closed the training-gap category. Sustained fix, not a meeting-decayed one.

HIGH

Updated test procedures

Locked down standard work at the test bay. Eliminated procedure drift.

MED

Weekly quality meetings

Cadence for issues to surface and be actioned. Without this the loop does not land.

MED

Weekly spot checks

Verifies procedures are being followed. Early-warning if drift returns.

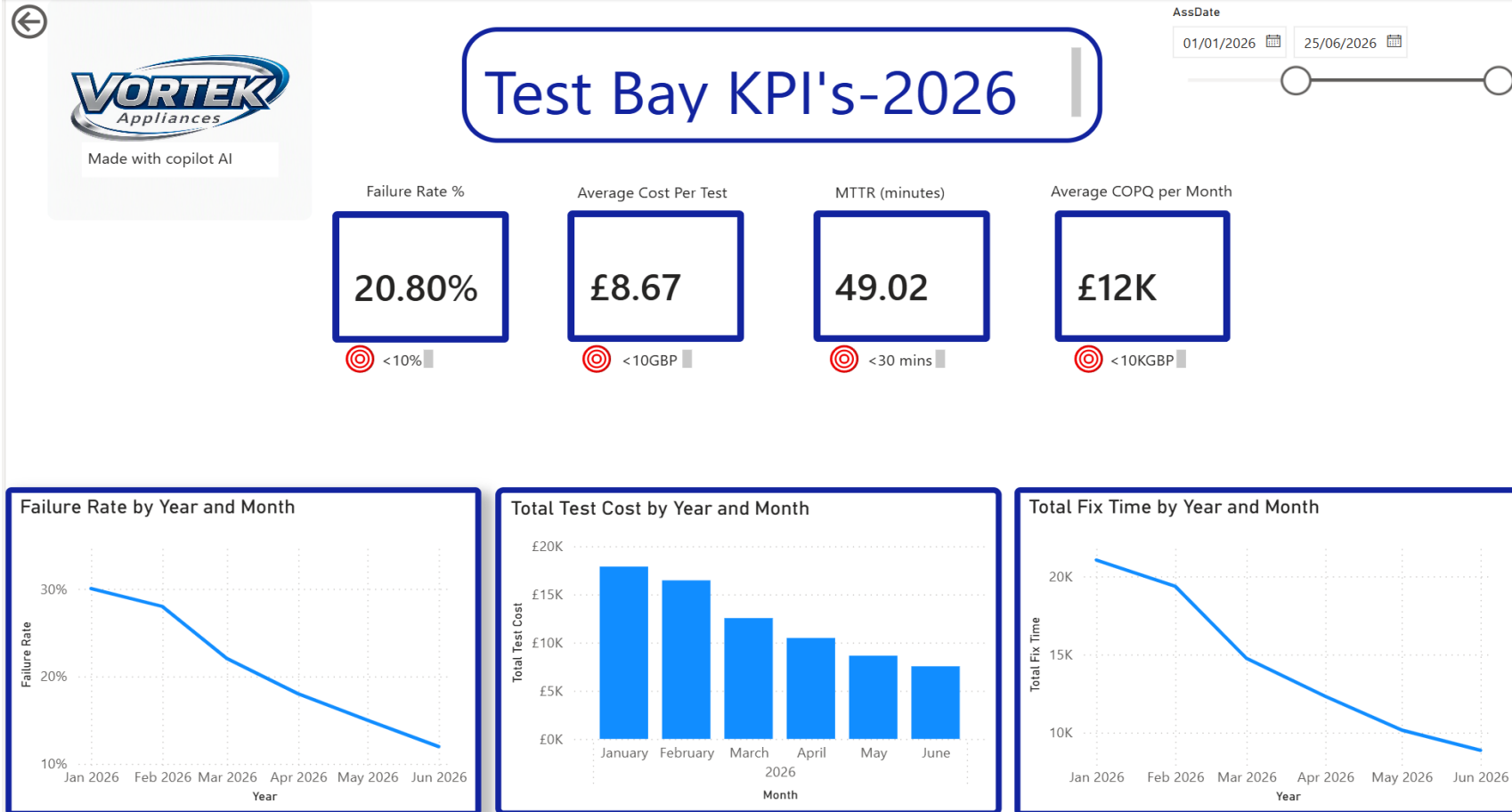
ENABLER

AI-generated reports

The enabler. Cut reporting time so hours went to root cause, not formatting.



Delivery - Test Bay KPI dashboard



EXECUTIVE KPIs

Failure Rate %

20.80%

Target <10%

Avg Cost/Test

£8.67

Target <£10

MTTR (mins)

49.02

Target <30 mins

Avg COPQ / month

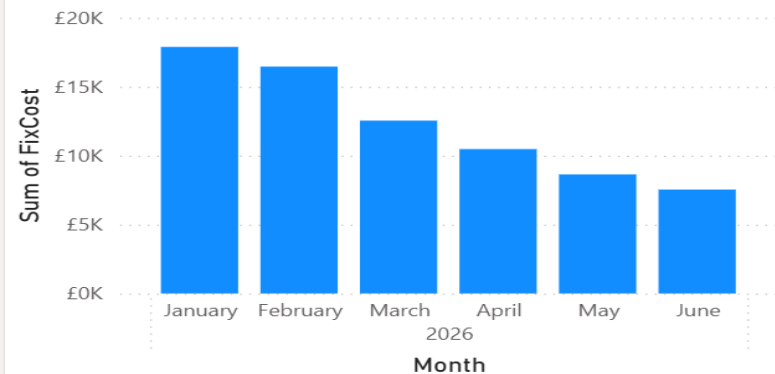
£12K

Target <£10K

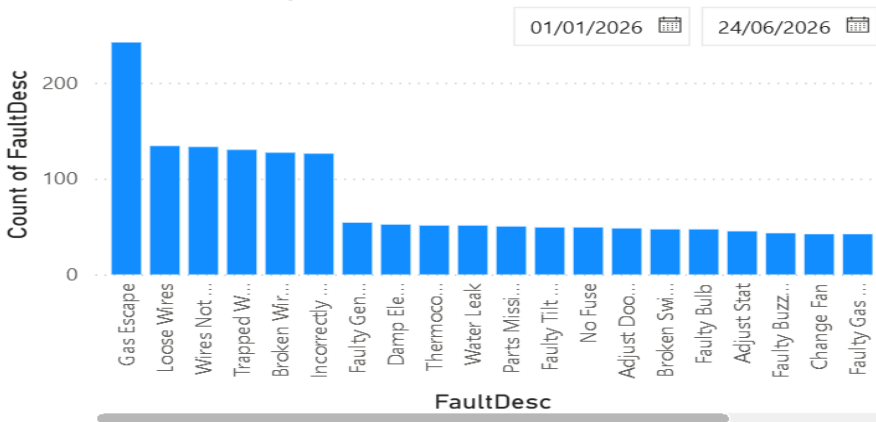


Delivery - overall summary dashboard

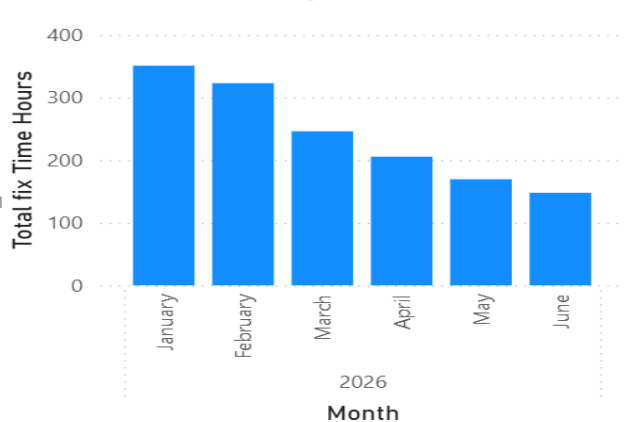
Sum of FixCost by Year and Month



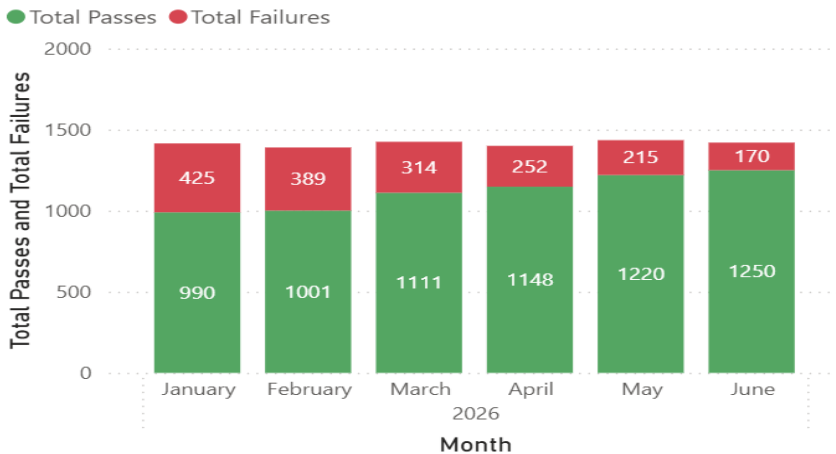
Count of FaultDesc by FaultDesc



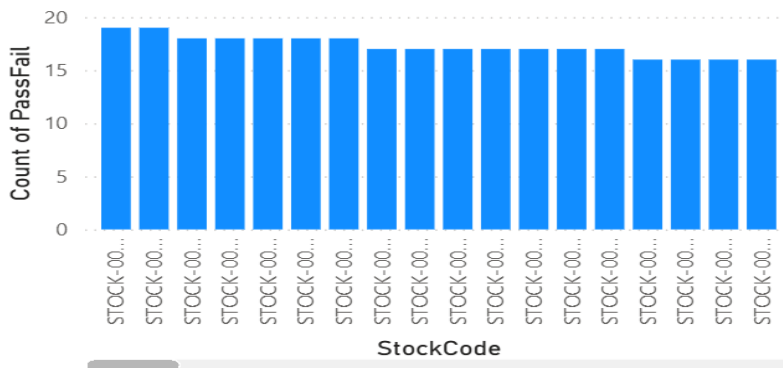
Total fix Time Hours by Year



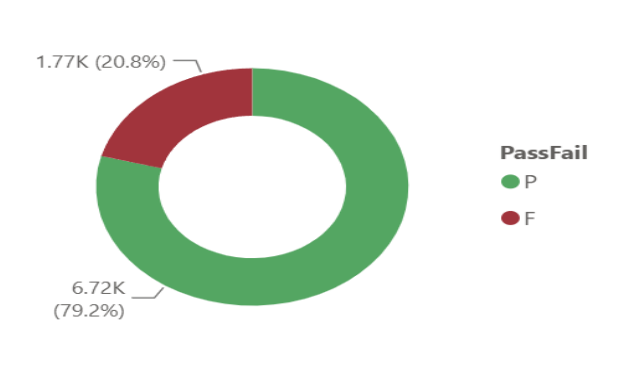
Total Passes and Total Failures by Year and Month



Count of PassFail by StockCode



Count of PassFail by PassFail



Cost, hours, fault mix and pass/fail - one page, six visuals, executive-ready.



Result - the numbers moved

BEFORE

Oct - Dec 2025

32.6%

failure rate

£20K peak monthly rework cost

3 months out of control (above UCL)

One in three units failing

AFTER

Apr - Jun 2026

15.0%

failure rate

£8K monthly rework cost

3 months below LCL - sustained

One in seven - a step-change



Considered and rejected - analytical honesty

Fitting the wrong model is worse than not fitting one.

X

Time-series forecasting

Only 9 months of data and a deliberate step-change in Feb 2026 breaks stationarity. ARIMA and Prophet would misfit either regime.

X

Row-level Pass/Fail classification

Dominant predictor would be a pre/post-intervention flag - descriptive, not predictive. Model rediscovers what we already know.

X

NLP on Comments field

Comments column is templated boilerplate from the fault code - NLP would re-derive FaultDesc. Would add real value only on genuine free-text engineer notes.

Inference over prediction. The analytical value here is proving the change worked, not guessing what happens next.



Next steps - scale, rigour, close the loop

1

Star-schema migration

fact_test + fact_fault (unpivoted) with dim_date, dim_fault (categorised), dim_tester, dim_assembler, dim_stock, dim_unit_type. Faster refresh, native fault-category slicing.

2

Row-level predictive model

Random forest on Pass/Fail with UnitType, Assembler, StockCode, Month features. Per-unit risk score surfaced on the dashboard.

3

Automated SPC monitoring

Scheduled refresh flags a month the moment it breaches a control limit. Alerts to Ops via Teams or Slack.

4

Intervention log integration

Small table (or Jira/ServiceNow feed) that pins each SPC change to a documented corrective action. Closes cause-to-effect loop.

5

Data-quality hardening

Fix currency encoding, lock date format, add NOT NULL and FK constraints once the star schema exists.



Takeaway

End-to-end quality analytics - pipeline built, story told, numbers moved.

- v Pipeline - SQL to Python to Power BI, end-to-end, repeatable, defensible
- v Quality engineering - Pareto + SPC + MTTR + COPQ used correctly, not as buzzwords
- v Audience awareness - Dashboards designed for FD, Ops, Tech and MD decisions



Thank you.

Joseph Kennedy - Quality and Operations Analytics - Portfolio 2026

Vortek Appliances is a fictional alias for an anonymised real-world manufacturer.